

Q1
Multiple Threads Using a Single
Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Topic: Thread class, multiple threads, if-else

Question:

Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.

Armstrong → Check whether the number is an Armstrong number.

Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number

Armstrong Thread: 153 is an Armstrong number

Reverse Thread: Reverse of 153 = 351

Your Code

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) throws InterruptedException {
5         Scanner sc = new Scanner(System.in);
6         int number = sc.nextInt();
7
8         MultiTask primeThread = new MultiTask("Prime", number);
9         MultiTask armstrongThread = new MultiTask("Armstrong", number);
10        MultiTask reverseThread = new MultiTask("Reverse", number);
11
12        primeThread.start();
13        primeThread.join();
14
15        armstrongThread.start();
16        armstrongThread.join();
17
18        reverseThread.start();
19        reverseThread.join();
20    }
21 }
```

Input

153

Output

```
Prime Thread: 153 is not a prime
number
Armstrong Thread: 153 is an
Armstrong number
Reverse Thread: Reverse of 153 =
351
```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10/10 / 10 marks

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Question:

Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.

FactorialThread → Find the factorial of the given number.

ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.

Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number

Factorial: 7! = 5040

Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

Your Code

```
1 import java.util.Scanner;  
2  
3 public class Main {  
4     public static void main(String[] args) throws InterruptedException  
5     {  
6         Scanner sc = new Scanner(System.in);  
7         int n = sc.nextInt();  
8         sc.close();  
9  
10        PrimeThread pt = new PrimeThread(n);  
11        FactorialThread ft = new FactorialThread(n);  
12        ReverseThread rt = new ReverseThread(n);  
13  
14        pt.start();  
15        pt.join();  
16  
17        ft.start();  
18        ft.join();  
19  
20        rt.start();  
21        rt.join();  
22    }  
23 }
```

Input

7

Output

```
Prime: 7 is a prime number  
Factorial: 7! = 5040  
Reverse: Reverse of 7 = 7
```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

20.00 / 20 marks

Final Count = 2000

Requirements:

Use a shared Counter object.

Both threads must access the same Counter object.

Use synchronized for the increment operation.

Use join() to ensure both threads finish before displaying the final count.

Your Code

```
1 class Counter {
2     int count = 0;
3
4     synchronized void increment() {
5         count++;
6     }
7
8     public static void main(String[] args) throws Exception {
9         Counter counter = new Counter();
10
11         ThreadA threadA = new ThreadA(counter);
12         ThreadB threadB = new ThreadB(counter);
13
14         threadA.start();
15         threadB.start();
16
17         threadA.join();
18         threadB.join();
19
20         System.out.println("ThreadA completed");
21         System.out.println("ThreadB completed");
22         System.out.println("Final Count = " + counter.count);
23     }
24 }
25
26 class ThreadA extends Thread {
27     Counter counter;
28
29     ThreadA(Counter counter) {
30         this.counter = counter;
31     }
```

Input

stdin input

Output

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Visible Test Cases

Test Case 1	✓
Test Case 2	✓
Test Case 3	✓

10.00 / 10 marks